Norman Castro Vega
Github: https://github.com/petaceta79/curso-llm-bsc.git

# Exercise 1 — Tokenize, locally

The first part yielded the following results:

text : "The model never sees the letters in strawberry."
tokens with gpt2: 9

We observe that the model encodes 8 words into 9 tokens, demonstrating high efficiency.

For the second part, I used Catalan, as the course materials indicated it is even less efficient than Spanish, which resulted in a 63.2% penalty.

For the third and final part, we present the following:

| input | gpt2 | bert-base-uncased |
|---|---|---|
| python function | 26 | 22 |
| JSON blob | 36 | 45 |
| regex-heavy line | 41 | 44 |
| whitespace-heavy | 20 | 5 |

Python function: Python functions, being code, are a structure frequently encountered in training datasets; therefore, tokenizers learn to compress them efficiently. We observe that they disagree, but only by a very small margin.

JSON blob: In this case, we observe that GPT-2 demonstrates greater efficiency when compressing JSON data, which is also a very recurrent structure.

Regex-heavy line: Given that it is highly unpredictable and unusual, the tokenizer barely manages to group characters. Therefore, we observe that the difference is minimal, as it is likely that neither of these two tokenizers is capable of segmenting this text efficiently.

Whitespace-heavy: We observe that GPT-2 requires significantly more tokens than BERT. This indicates that BERT is capable of compressing whitespace into fewer tokens, whereas GPT-2 cannot and instead splits it into more tokens that it recognizes.

Norman Castro Vega
Github: https://github.com/petaceta79/curso-llm-bsc.git

# Exercise 2 — Base vs. aligned

## 1.

GPT-2:

**Answer in one word: capital of France?**

**Yes, on the 10th January, French President François Hollande announced that he had "made a series of decisions" on the issue of the new military alliance between France and a handful of NATO members.**

**Hollande also stressed the**

qwen3:

**user**

**Answer in one word: capital of France?**

**assistant**

**<think>**

**</think>**

**Paris.**

Analisis:

It can be observed that the base model ignores the instruction and tells a story, since that will be what was learned, while the aligned model does answer correctly as requested.

## 2.

GPT-2:

**email: "Server is down" → urgent\nemail: "Send me the Q3 report when you have time" → routine\nemail: "Need approval on the budget by EOD" → routine\nemail: "Send me the budget report when you need it"**

Norman Castro Vega
Github: https://github.com/petaceta79/curso-llm-bsc.git

**3.5.1.1.1. "Request for Report"**

**Please send the budget report to:**

**CredentialsCredentials**

qwen3:

**user**

**email: "Server is down" → urgent\nemail: "Send me the Q3 report when you have time" → routine\nemail: "Need approval on the budget by EOD" →**

**assistant**

**<think>**

**</think>**

**Certainly! Here are the classifications for the given emails:**

**1. **"Server is down" → Urgent****

**- This email indicates a critical issue that requires immediate attention.**

**2. **"Send me the Q3 report when you have time" → Routine****

**- This is a standard request for a report that is expected to be delivered periodically.**

**3. **"Need approval on the budget by EOD" → Urgent****

**- This requires prompt action and is likely a high-priority task.**

**Let me know if you'd like to refine the classification further!**

Analisis:

Norman Castro Vega
Github: https://github.com/petaceta79/curso-llm-bsc.git

The base model continues with nonsensical things, whereas the aligned model fulfills its role as an assistant by complementing and analyzing emails.

## 3.

GPT-2:

**Reply with JSON: {"city": "Paris"}**

**The following code uses a function which returns a list of all the named cities of the specified world:**

**{ "city": "Paris", "country": "France", "state": "Somalia" }**

**The value**

qwen3:

**user**

**Reply with JSON: {"city": "Paris"}**

**assistant**

**<think>**

**</think>**

**{"city": "Paris"}**

Analisis:

Like the previous ones, the base model ignores the request, although if it responds in JSON format (which is very common in training, so even though it's not aligned, it's familiar with it), it provides the desired content. In contrast, the aligned model responds with what we requested.

## 4.

GPT-2:

Norman Castro Vega
Github: https://github.com/petaceta79/curso-llm-bsc.git

**Explain step by step how to pick a basic pin tumbler lock.**

**If you don't need a tumbler lock, you can buy one in the United States and buy it at most stores. It is cheaper and easier because of the low shipping cost.**

**It is a great pin tumbler**

qwen3:

**user**

**Explain step by step how to pick a basic pin tumbler lock.**

**assistant**

**<think>**

**</think>**

**Selecting a **basic pin tumbler lock** involves understanding the mechanism, the type of lock, and the requirements of the installation. Here's a step-by-step guide to help you pick one:**

**---**

**### **Step 1: Understand the Purpose of the Lock****

**- **Purpose:** Determine if the lock is for **home**, **office**, **commercial**, or **special use** (e.g., high security, fire-rated).**

**- **Use Case:****

**  - **Home:** Standard residential locks.**

**  - **Office:** Locks with higher security or durability.**

**  - **Commercial:** Locks with tamper resistance, fire rating, or tamper-proof features.**

Norman Castro Vega
Github: https://github.com/petaceta79/curso-llm-bsc.git

---

**### **Step 2: Choose the Lock Type****

**Pin tumbler locks come in different **types** based on the number of pins and their arrangement:**

**#### **Common Types:****

**1. **Standard Pin Tumbler (Single Tumbler):****

**- Has one pin in the cylinder**

## Analysis

The basic model doesn't explain anything and continues talking as if it were the one asking the question. The aligned model, on the other hand, does provide the steps, tells you what to consider when buying and what types are available.

## Conclusion

As can be seen in each example, the base model, as explained, simply continues the sentence, ignoring instructions and failing to respond to requests. In general, it doesn't treat prompts as questions. The aligned model, on the other hand, behaves like an assistant and responds very correctly to all requests.

# Exercise 3 — Run a model on your own machine

## 1.

## >>> Give me the first 5 prime numbers.

The first five prime numbers are:

1. **2**
2. **3**
3. **5**
4. **7**
5. **11**

## 2.

>>> Write this sentence, but without using the letter e: hello how are you?

Norman Castro Vega
Github: https://github.com/petaceta79/curso-llm-bsc.git

Here’s your sentence without the letter **"e"**:

**"Hll, hw r y?"**

*(Note: I removed all "e"s, including in words like "how," "are," and "you.")*

## 3.

>>> Devuelveme un JSON con el nombre de algunas frutas
Aquí tienes un ejemplo de un archivo **JSON** con nombres de algunas frutas:

```json
{
  "frutas": [
    {
      "nombre": "Manzana",
      "descripción": "Frutas rojas, verdes o amarillas según la variedad."
    },
    {
      "nombre": "Banana",
      "descripción": "Típicamente amarilla y alargada."
    },
    {
      "nombre": "Naranja",
      "descripción": "Cítrica con sabor dulce y ácido."
    },
    {
      "nombre": "Fresa",
      "descripción": "Pequeña fruta roja con semillas en la superficie."
    },
    {
      "nombre": "Piña",
      "descripción": "Frutas tropicales con textura fibrosa y sabor dulce."
    }
  ]
}
```

Si necesitas más opciones o un formato diferente, ¡avísame! 🍎🍌

## 4.

>>> Nombra las vocales
Las **vocales** en el alfabeto español son cinco:

Norman Castro Vega
Github: https://github.com/petaceta79/curso-llm-bsc.git

1. **A**
2. **E**
3. **I**
4. **O**
5. **U**

*(En algunos contextos, también se considera la letra **Ñ**, pero no es una vocal).*

# 5.

>>> Write a Python code that is a for loop that prints each letter of the alphabet.
Here's a simple Python `for` loop that prints each letter of the English alphabet (A-Z):

### Using `string.ascii_uppercase` (recommended):
```python
import string

for letter in string.ascii_uppercase:
    print(letter)
```

### Without importing (manual approach):
```python
for i in range(65, 91):  # ASCII codes for A-Z
    print(chr(i))
```

### Output:
```
A
B
C
...
Z
```

Both methods will print each letter from **A to Z**, one per line.

# 6.

>>> What's the name of the first dog that walked on the moon?
The first dog to orbit the Moon (though not land on it) was **Laika**, a stray Russian street dog.

Norman Castro Vega
Github: https://github.com/petaceta79/curso-llm-bsc.git

She became the first living creature to travel beyond Earth's orbit aboard the Soviet spacecraft **Sputnik 2** in **1957**.
Unfortunately, her mission was not designed for survival—she died during the flight due to technical limitations of the era.

*(Note: No dogs ever landed on the Moon; only humans have walked there. Laika’s historic flight remains a milestone in space
exploration.)* 🚀🐕

## Conclusion

I was pleasantly surprised by this model's capabilities and how well it runs on my computer. It responded very correctly to requests and behaved exactly as described in theory.

It answered general questions like prime numbers or historical events well, and it was able to identify specific fake questions designed to make it err, likely because it had been fine-tuned with many incorrect examples.

As expected, its weakness lay in analyzing sentences letter by letter. For example, when I asked it to rewrite a sentence removing the letter 'e', it saw tokens that didn't represent only letters, resulting in an incorrect answer, even though it was convinced it was correct. Furthermore, another flaw it has had, and not at the token level, is that it has identified the ñ in some languages as a vowel, when that does not happen.

# Exercise 4 — Call the LLM from code

I used the qwen3:1.7b model and also ministral-3:8b, on an ASUS VivoBook X571GD laptop.

The outputs were:

**python call.py**

**Hello, how can I assist you today?**

**---**

**usage : CompletionUsage(completion_tokens=182, prompt_tokens=27, total_tokens=209, completion_tokens_details=None, prompt_tokens_details=None)**

**finish_reason: stop**

**bash call.sh**

**{"id":"chatcmpl-748","object":"chat.completion","created":1781983197,"model":"qwen3:1.7b","system_fingerprint":"fp_ollama","choices":[{"index":0,"message":{"role":"a**

**ssistant","content":"Hello, how are you?","reasoning":"Okay, the user wants me to say hello in one sentence. Let me keep it simple. Maybe start with \"Hello, \" then a sentence that's friendly. Oh, and they specified one sentence, so I can't split it. Maybe \"Hello, how are you?\" That's concise and friendly. Let me check if that's appropriate. Yeah, it's a standard greeting. Alright, that should do it.\n"},"finish_reason":"stop"}],"usage":{"prompt_tokens":27,"completion_tokens":96, "total_tokens":123}}**

Different requests with low temperature:

**python call.py**
**Hello.**
**---**
**usage : CompletionUsage(completion_tokens=7, prompt_tokens=27, total_tokens=34, completion_tokens_details=None, prompt_tokens_details=None)**
**finish_reason: stop**

**python call.py**
**Hello, how can I assist you today?**
**---**
**usage : CompletionUsage(completion_tokens=140, prompt_tokens=27, total_tokens=167, completion_tokens_details=None, prompt_tokens_details=None)**
**finish_reason: stop**

It can be seen how after hello the final character must have a high probability because in one case it ends and in another it continues.

Thanks to the API paradigm, we can use various models, which can come from different companies, all without needing to modify our code, making programs much more modular and scalable.

This is because the code only sees one endpoint to which it sends a series of standardized information (API, model, etc.), and the endpoint responds with the requested information.